CFD Assignment 2

Francesco James Marino
Imperial College London, UK

January 24, 2026

1 Task 1

As shown in fig. 1 Initially, we can see vorticity being created at the cylinders, as expected. By $nt = 7500$, we see a vortex street being created behind the cylinder and re-entering the flow from the opposite boundary in front of the cylinder, simulating a periodic infinite grid of cylinders.

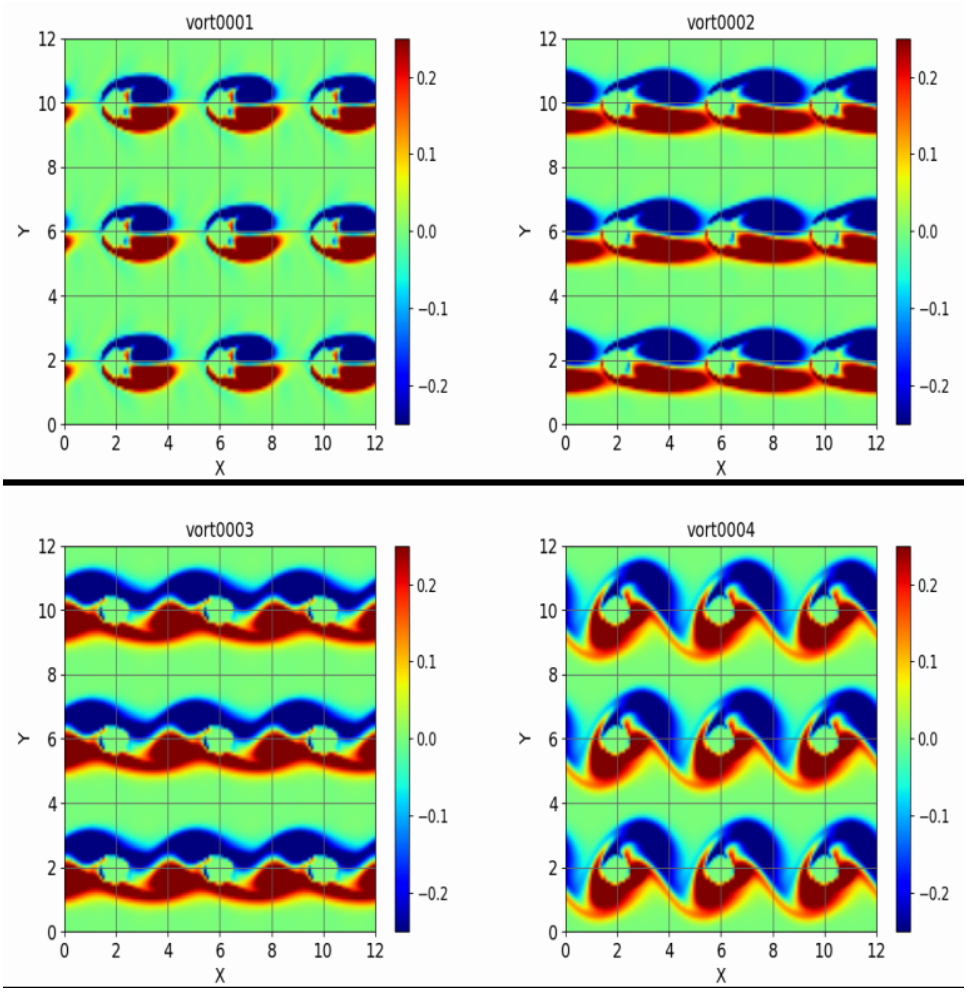

Figure 1: Visualisation of vorticity fields for $nt = 2500, 5000, 7500, 10000$, solved using a centred difference scheme in space and a second-order Adams-Bashforth scheme in time

2 Task 2

The compressible Navier-Stokes equations are a combination of a hyperbolic advection term, which dominates at high Reynolds numbers, and a parabolic viscous term.

Analysing the 1D linear advection equation (representing the linearised hyperbolic sub-problem), discretised using a centred difference scheme in space and the second-order Adams-Bashforth scheme in time, using Von Neumann stability analysis, we can easily find that it is unconditionally unstable. In the full system, numerical stability is maintained at low CFL numbers by viscous dissipation. However, at a CFL of 0.75, the increased time step renders this damping insufficient to counteract the inviscid instability leading to exponential error growth and solver divergence.

3 Task 3

I implemented the subroutine rkutta as follows:

```
    subroutine rkutta(rho,rou,rov,roe,fro,gro,fru,gru,frv,grv,&
     fre,gre,nx,ny,ns,dlt,coef,k)
!
!############################################################

  implicit none
!
  real(8),dimension(nx,ny) :: rho,rou,rov,roe,fro,gro,fru,gru,frv
  real(8),dimension(nx,ny) :: grv,fre,gre
  real(8),dimension(2,ns) :: coef
  real(8) :: dlt
  integer :: i,j,nx,ny,ns,k
!
!coefficient for RK sub-time steps
        coef(1,1)=8.0/15.0*dlt
        coef(1,2)=5.0/12.0*dlt
        coef(1,3)=3.0/4.0*dlt
        coef(2,1)=0.0
        coef(2,2)=-17.0/60.0*dlt
        coef(2,3)=-5.0/12.0*dlt



  do j=1,ny
     do i=1,nx
      rho(i,j)=rho(i,j)+coef(1,k)*fro(i,j)+coef(2,k)*gro(i,j)
      gro(i,j)=fro(i,j)
      rou(i,j)=rou(i,j)+coef(1,k)*fru(i,j)+coef(2,k)*gru(i,j)
      gru(i,j)=fru(i,j)
      rov(i,j)=rov(i,j)+coef(1,k)*frv(i,j)+coef(2,k)*grv(i,j)
      grv(i,j)=frv(i,j)
      roe(i,j)=roe(i,j)+coef(1,k)*fre(i,j)+coef(2,k)*gre(i,j)
      gre(i,j)=fre(i,j)
     enddo
  enddo

  return
end subroutine rkutta
```

As shown in fig. 2, The third-order Runge-Kutta scheme scheme successfully stabilises the simulation at $CFL = 0.75$. This is because by taking 4 substeps, we are increasing our computational domain of dependence, which hence contains the physical domain of dependence and stabilises the scheme. We can see that the fundamental physics of vortex shedding remains consistent with what is seen in Task 1; however, because Δt is now three times larger, we can see the flow developing three times as fast. Furthermore, the higher-order temporal accuracy of this scheme provides better preservation of the vortex flow as they transport downstream.

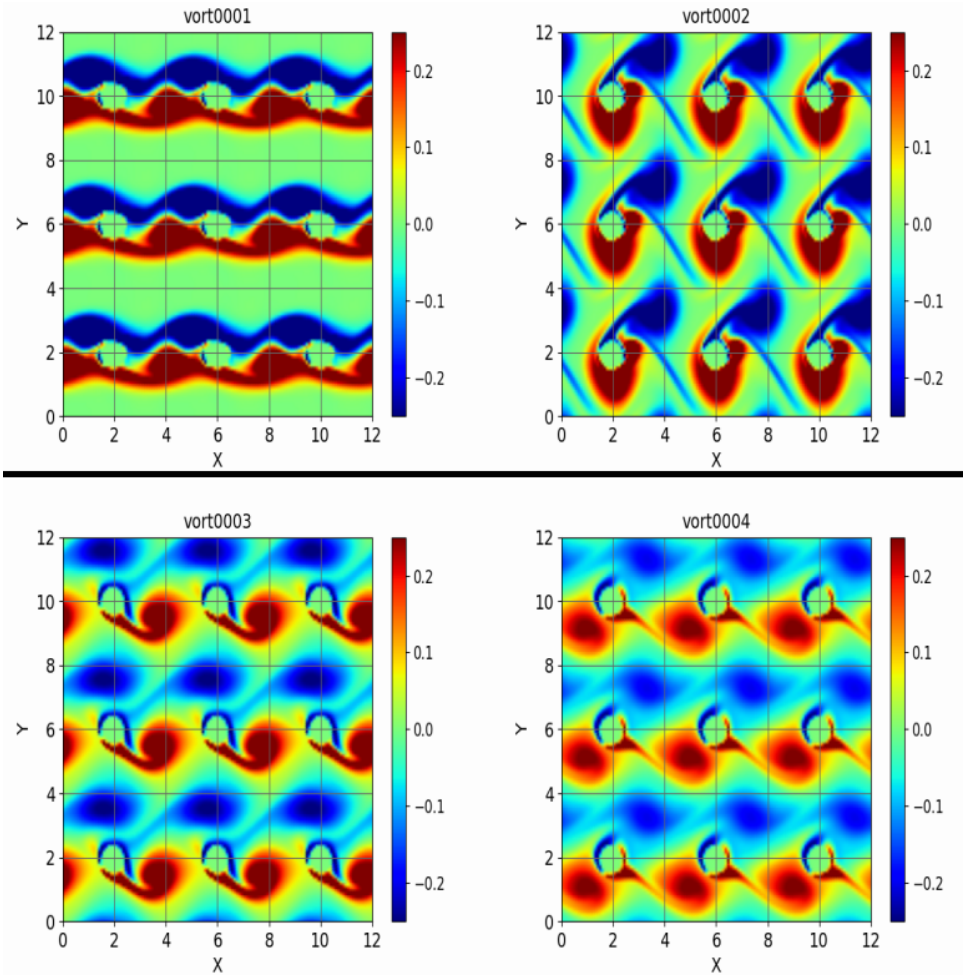

Figure 2: Visualisation of vorticity fields for nt $= 2500, 5000, 7500, 10000$*, solved using a centred difference scheme in space and a third-order Runge-Kutta scheme in time*

4 Task 4

I implemented the subroutines *derix4, deriy4, derxx4 and deryy4* as follows:

derix4:

```
    subroutine derix4(phi,nx,ny,dfi,xlx)
!
!Fourth-order first derivative in the x direction
!###############################################

  implicit none

  real(8),dimension(nx,ny) :: phi,dfi
  real(8) :: dlx,xlx,udx
  integer :: i,j,nx,ny,im1,im2,ip1,ip2

   dlx=xlx/nx
   udx = 1.0 / (12.0 * dlx)

  do j=1,ny
     do i=1,nx
        im1 = i - 1; if (im1 < 1) im1 = im1 + nx
        im2 = i - 2; if (im2 < 1) im2 = im2 + nx
```

```
        ip1 = i + 1; if (ip1 > nx) ip1 = ip1 - nx
        ip2 = i + 2; if (ip2 > nx) ip2 = ip2 - nx

        dfi(i,j) = udx * (phi(im2,j) - 8.0*phi(im1,j) + 8.0*phi(ip1,j) - phi(ip2,j))
     enddo
  enddo


  return
end subroutine derix4
```

deriy4:

```
    subroutine deriy4(phi,nx,ny,dfi,yly)
!
!Fourth-order first derivative in the y direction
!############################################

  implicit none

  real(8),dimension(nx,ny) ::  phi,dfi
  real(8) :: dly,yly,udy
  integer :: i,j,nx,ny,jm1,jm2,jp1,jp2

  dly=yly/ny
  udy = 1.0 / (12.0 * dly)

  do j=1,ny
     jm1 = j - 1; if (jm1 < 1) jm1 = jm1 + ny
     jm2 = j - 2; if (jm2 < 1) jm2 = jm2 + ny
     jp1 = j + 1; if (jp1 > ny) jp1 = jp1 - ny
     jp2 = j + 2; if (jp2 > ny) jp2 = jp2 - ny
     do i=1,nx
        dfi(i,j) = udy * (phi(i,jm2) - 8.0*phi(i,jm1) + 8.0*phi(i,jp1) - phi(i,jp2))
     enddo
  enddo

  return
end subroutine deriy4
```

derxx4:

```
    subroutine derxx4(phi,nx,ny,dfi,xlx)
!
!Fourth-order second derivative in y direction
!############################################

  implicit none

  real(8),dimension(nx,ny) ::  phi,dfi
  real(8) :: dlx,xlx,udx
  integer :: i,j,nx,ny,im1,im2,ip1,ip2

  dlx=xlx/nx
  udx = 1.0 / (12.0 * dlx * dlx)

  do j=1,ny
     do i=1,nx
```

```
        im1 = i - 1; if (im1 < 1) im1 = im1 + nx
        im2 = i - 2; if (im2 < 1) im2 = im2 + nx
        ip1 = i + 1; if (ip1 > nx) ip1 = ip1 - nx
        ip2 = i + 2; if (ip2 > nx) ip2 = ip2 - nx

        dfi(i,j) = udx * (-phi(im2,j) + 16.0*phi(im1,j) - 30.0*phi(i,j) + 16.0*phi(ip1,j) - phi(ip2,j
     enddo
  enddo

  return
end subroutine derxx4
```

deryy4:

```
    subroutine deryy4(phi,nx,ny,dfi,yly)
!
!Fourth-order second derivative in the y direction
!##########################################

  implicit none

  real(8),dimension(nx,ny) ::  phi,dfi
  real(8) :: dly,yly,udy
  integer :: i,j,nx,ny,jm1,jm2,jp1,jp2
  dly=yly/ny
  udy = 1.0 / (12.0 * dly * dly)

  do j=1,ny
     jm1 = j - 1; if (jm1 < 1) jm1 = jm1 + ny
     jm2 = j - 2; if (jm2 < 1) jm2 = jm2 + ny
     jp1 = j + 1; if (jp1 > ny) jp1 = jp1 - ny
     jp2 = j + 2; if (jp2 > ny) jp2 = jp2 - ny
     do i=1,nx
        dfi(i,j) = udy * (-phi(i,jm2) + 16.0*phi(i,jm1) - 30.0*phi(i,j) &
                          + 16.0*phi(i,jp1) - phi(i,jp2))
     enddo
  enddo

  return
end subroutine deryy4
```

As shown in fig. 3, visually, the 4th-order results are similar to Task 1. However, by looking at the difference in vorticity between the two plots shown in fig. 4, we can see the fourth-order scheme significantly reduces numerical dispersion, especially in regions of large vorticity and hence where the spatial derivatives are largest, and thus where truncation errors are most prominent. This means in fig. 3, the vorticity concentrations appear slightly sharper, preserving the high-gradient shear layers more accurately.

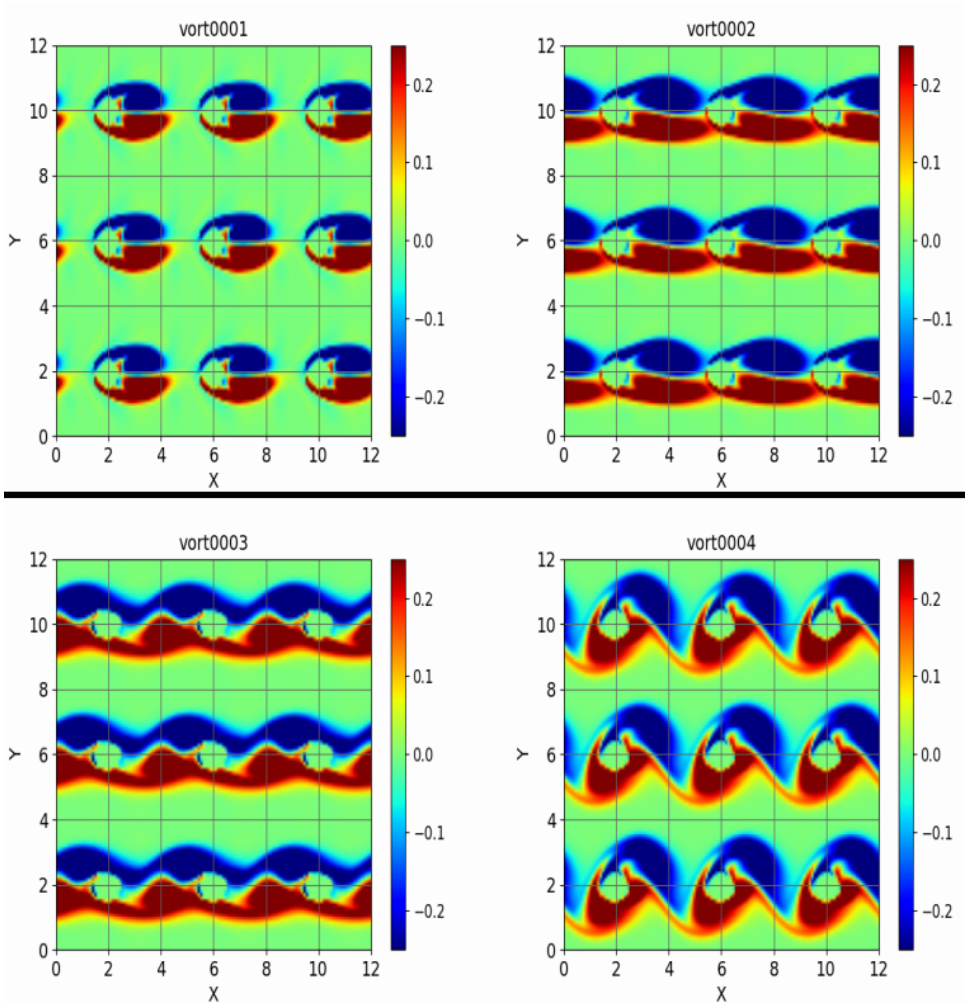

Figure 3: Visualisation of vorticity fields for $nt = 2500, 5000, 7500, 10000$*, solved using a fourth-order centred difference scheme in space and a second-order Adams-Bashforth scheme in time*

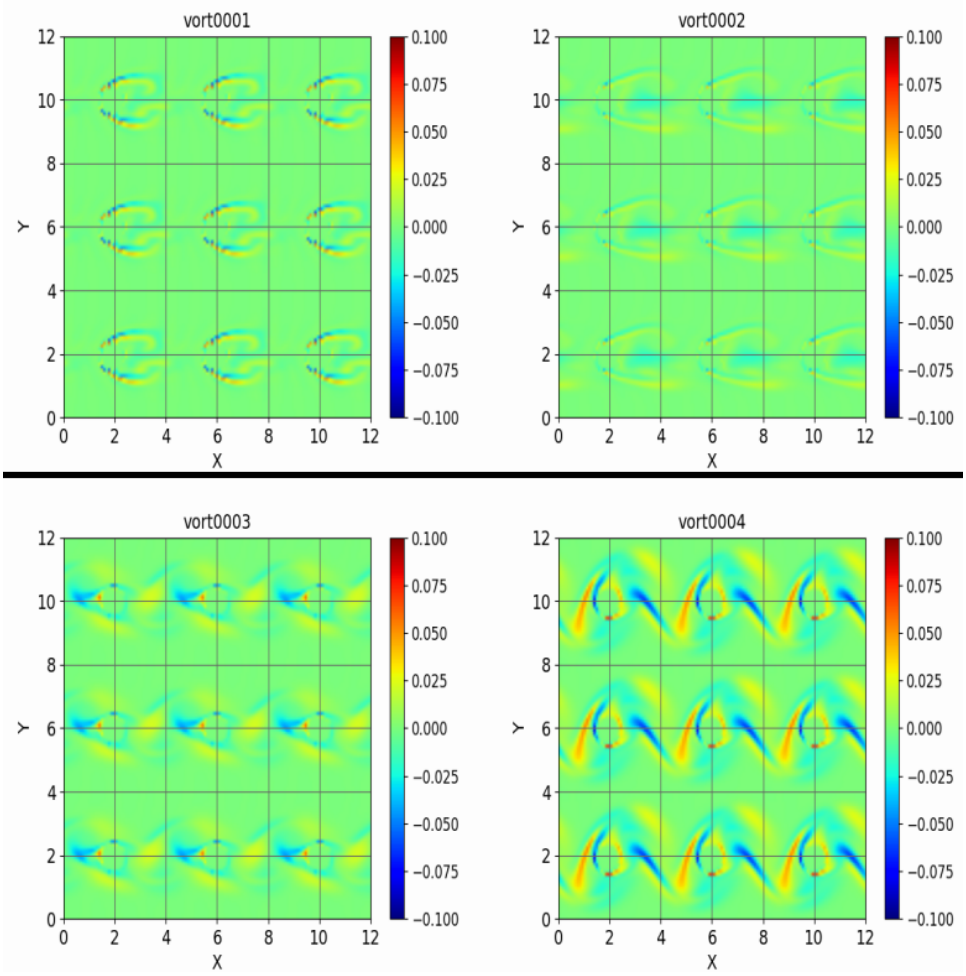

Figure 4: Visualisation of difference between vorticity fields for nt $= 2500, 5000, 7500, 10000$*, computed using a fourth-order centred difference scheme in space and a second-order centred difference scheme in space, both using a second-order Adams-Bashforth scheme in time*

5 Task 5

Simulating for a long time eventually leads to the vorticity becoming homogeneous throughout the whole domain. In fact, the periodic boundary conditions mean the flow exits one boundary and re-enters the other and thus will continue to collide with the cylinder, continuously producing vorticity. Since the flow is 2D, the vorticity can only be produced at the boundaries and then be convected and diffused, as it cannot bend or stretch. Furthermore, since the energy is limited to what it was in the initial conditions, there is only a finite amount of vorticity that can be produced, and it continuously diffuses such that eventually it all homogenises. A development towards what was just described is shown in fig. 5. after 10000 time steps.

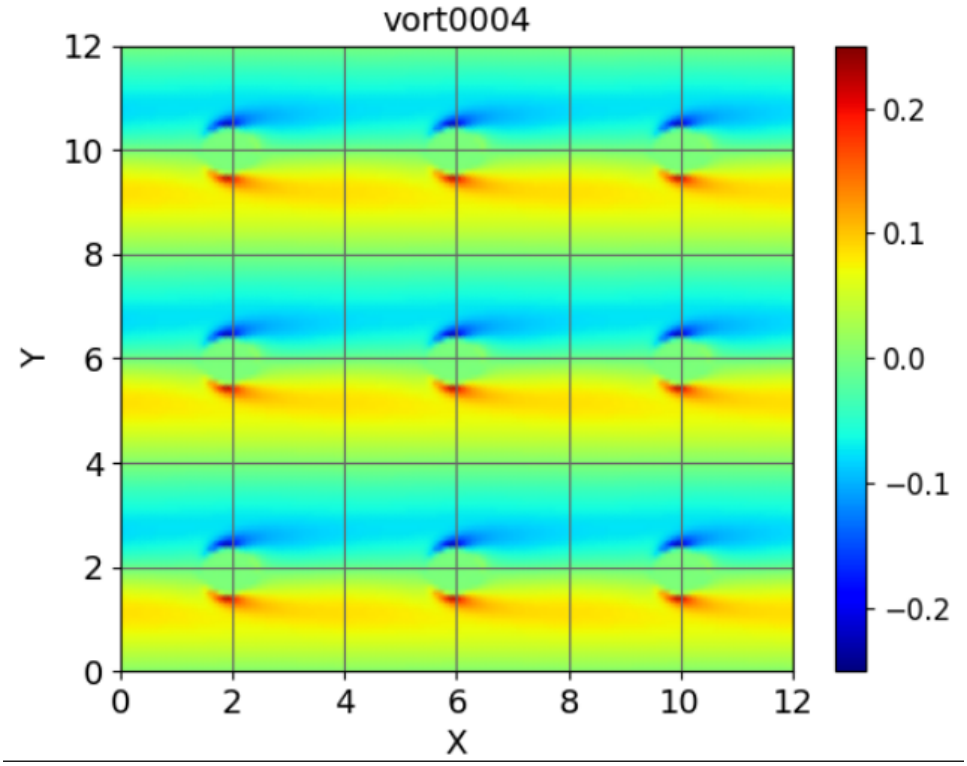

Figure 5: Visualisation of vorticity fields after 100000 *time steps, solved using a centred difference scheme in space and a second-order Adams-Bashforth scheme in time*

6 Task 6

To change the flow direction, the only change that needs to be made is the initial condition in the streamwise direction. For this, I changed lines 625 and 626 to

```
uu0=-mach*cci
xmu=roi*abs(uu0)*d/ren
```

which change the initial conditions to have a negative velocity (from right to left) but also keeping the dynamic viscosity a positive coefficient. The result is shown in fig. 6

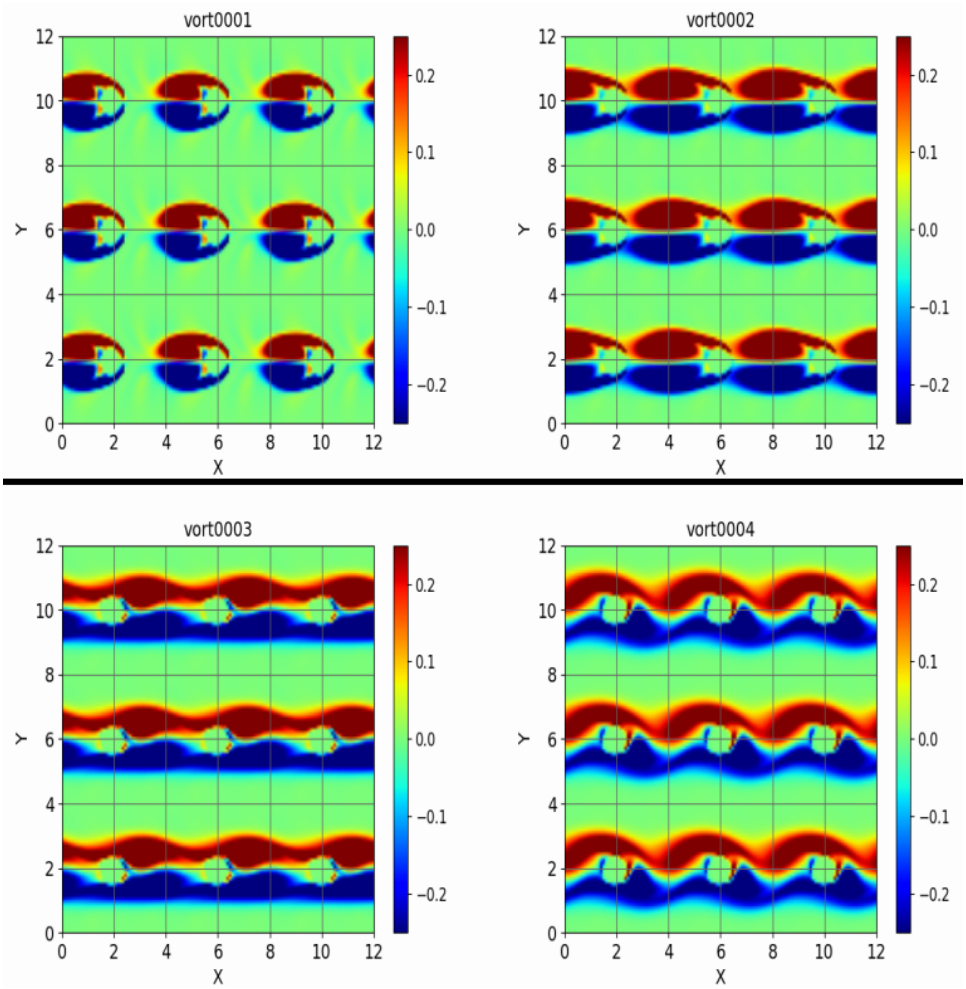

Figure 6: Visualisation of vorticity fields for nt $= 2500, 5000, 7500, 10000$*, solved using a centred difference scheme in space and a second-order Adams-Bashforth scheme in time with initial condition of streamwise flow from right to left*

However, to render the flow identically symmetrical at each time step to the one in task 1, we need to also keep the initial conditions symmetrical. Thus, we map the x coordinate also of the transverse velocity initial condition to $x' = xlx - x$, where xlx is the width of the periodic domain. This changes $\sin 4\pi x/xlx \to -\sin 4\pi x/xlx$ and $\sin 7\pi x/xlx \to \sin 7\pi x/xlx$. For this, I changed lines 585 to 567 to:

```
vvv(i,j)=0.01*(-sin(4.*pi*(i)*dlx/xlx)&
           +sin(7.*pi*(i)*dlx/xlx))*&
           exp(-(j*dly-yly/2.)**2)
```

The result is shown in fig. 7

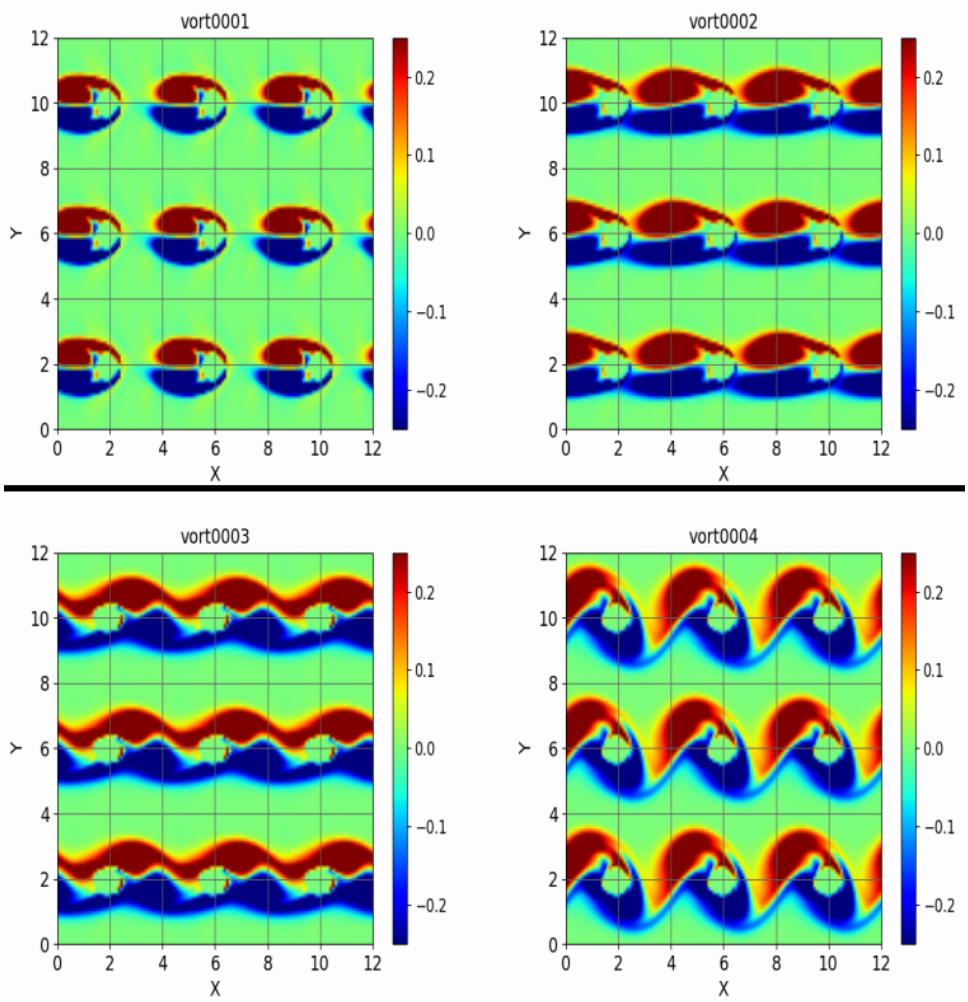

Figure 7: Visualisation of vorticity fields for nt $= 2500, 5000, 7500, 10000$*, solved using a centred difference scheme in space and a second-order Adams-Bashforth scheme in time with initial condition of flow from right to left for both u and v*

7 Task 7

To simulate the flow over a single cylinder I used Dirichlet conditions in the streamwise direction as the inlet boundary condition and a simple convective equation as the outflow condition. This allows wake vortices to exit the domain at the convection velocity u_0, which is a good enough approximation. In order to do so I first implemented a boundary routine which determines all the values at the inlet and outlet and then changed the subroutines derix and derxx in order to use values at the boundaries to calculate the derivatives near the boundary, getting rid of the periodicity in the streamwise direction.

The forementioned routines were written as follows:

boundary:

```
subroutine boundary(rho,rou,rov,roe,nx,ny,uu0,dlt,dx,xlx,yly,xmu,xba,gma)

  implicit none
  integer :: nx, ny, j
  real(8) :: xlx,yly,xmu,xba,gma,roi,cci,d,tpi,chv,uu0
  real(8), dimension(nx,ny) :: rho,rou,rov,roe
  real(8) :: dlt, dx, c_conv

  call param(xlx,yly,xmu,xba,gma,roi,cci,d,tpi,chv,uu0)
```

```
  c_conv = (uu0 * dlt) / dx ! convective speed at outflow

  do j=1,ny
   ! INFLOW (x=1): Dirichlet BC
     rho(1,j) = roi
     rou(1,j) = roi*uu0
     rov(1,j) = 0.
     roe(1,j) = roi*(chv*tpi+0.5*uu0*uu0)

     ! OUTFLOW (x=nx): simple convective boundary condition
     rho(nx,j) = rho(nx,j) - c_conv * (rho(nx,j) - rho(nx-1,j))
     rou(nx,j) = rou(nx,j) - c_conv * (rou(nx,j) - rou(nx-1,j))
     rov(nx,j) = rov(nx,j) - c_conv * (rov(nx,j) - rov(nx-1,j))
     roe(nx,j) = roe(nx,j) - c_conv * (roe(nx,j) - roe(nx-1,j))

  enddo

  end subroutine boundary
```

derx:

```
subroutine derix(phi,nx,ny,dfi,xlx)
!
!First derivative in the x direction
!###########################################

  implicit none

  real(8),dimension(nx,ny) :: phi,dfi
  real(8) :: dlx,xlx,udx
  integer :: i,j,nx,ny

  dlx=xlx/nx
  udx=1./(dlx+dlx)
  do j=1,ny
     dfi(1,j)=udx*(-1.*phi(3,j)+4.*phi(2,j)-3.*phi(1,j)) ! one-sided second-order difference at the b
     do i=2,nx-1
        dfi(i,j)=udx*(phi(i+1,j)-phi(i-1,j))
     enddo
     dfi(nx,j)=udx*((1.*phi(nx-2,j)-4.*phi(nx-1,j)+3.*phi(nx,j))) ! one-sided second-order difference
  enddo

  return
end subroutine derix
```

and **derxx**:

```
  subroutine derxx(phi,nx,ny,dfi,xlx)
  !
  !Second derivative in y direction
  !###########################################

  implicit none

  real(8),dimension(nx,ny) ::  phi,dfi
  real(8) :: dlx,xlx,udx
  integer :: i,j,nx,ny
```

```
  dlx=xlx/nx
  udx=1./(dlx*dlx)
  do j=1,ny
     dfi(1,j)=udx*(2*phi(1,j)-5*phi(2,j)+ 4*phi(3,j)-phi(4,j)) ! one-sided 2nd order difference at th
     do i=2,nx-1
        dfi(i,j)=udx*(phi(i+1,j)-(phi(i,j)+phi(i,j))&
             +phi(i-1,j))
     enddo
     dfi(nx,j)=udx*(2*phi(nx,j)-5*phi(nx-1,j)+ 4*phi(nx-2,j)-phi(nx-3,j)) ! one-sided 2nd order diffe
  enddo

  return
end subroutine derxx
```

Furthermore, in order to allow the boundary routine to update the edges independently, I only updated the points going from 2 to $nx - 1$ in the adams subroutine. Here the new adams subroutine:

```
subroutine adams(rho,rou,rov,roe,fro,gro,fru,gru,frv,grv,&
     fre,gre,nx,ny,dlt)
!
!############################################################

  implicit none

  real(8),dimension(nx,ny) :: rho,rou,rov,roe,fro,gro,fru,gru,frv
  real(8),dimension(nx,ny) :: grv,fre,gre
  real(8) :: dlt,ct1,ct2
  integer :: nx,ny,i,j

  ct1=1.5*dlt
  ct2=0.5*dlt
  do j=1,ny
     do i=2,nx-1
        rho(i,j)=rho(i,j)+ct1*fro(i,j)-ct2*gro(i,j)
        gro(i,j)=fro(i,j)
        rou(i,j)=rou(i,j)+ct1*fru(i,j)-ct2*gru(i,j)
        gru(i,j)=fru(i,j)
        rov(i,j)=rov(i,j)+ct1*frv(i,j)-ct2*grv(i,j)
        grv(i,j)=frv(i,j)
        roe(i,j)=roe(i,j)+ct1*fre(i,j)-ct2*gre(i,j)
        gre(i,j)=fre(i,j)
     enddo
  enddo

  return
end subroutine adams
```

I also changed the geometry of the simulation by changing the domain size to $20d12d$, with $nxny = 513257$ mesh nodes to ensure the wake has sufficient space to develop and exit without side-wall interference, and by changing the position of the centre of the circular to $(5d, 6d)$.

In order to do this, I change the following parts of code: **beginning of program navierstokes**:

```
  integer,parameter :: nx=513,ny=257,nt=10000,ns=3
```

Cylinder position in subroutine initl:

```
do j=1,ny
   do i=1,nx
      if (((i*dlx-5.*d)**2+(j*dly-yly/2.)**2).lt.radius**2) then
         eps(i,j)=1.
      else
         eps(i,j)=0.
      end if
   enddo
enddo
```

Finally, I printed the values of the vorticity field in the domain at $nt = 5500$ and $nt = 50000$, chosen to ensure the vortex street already reached the developed periodic flow, and called the boundary subroutine in the main program before the adams subroutine was called, in order to calculate the values at the outflow boundaries based on the values at $i = nx - 1$ from the cuurent time-step, of variables both at $i = nx$, which update only in the boundary subroutine, and at $i = nx - 1$, which update in the adams subroutine:

```
call fluxx(uuu,vvv,rho,pre,tmp,rou,rov,roe,nx,ny,tb1,tb2,tb3,tb4, &
     tb5,tb6,tb7,tb8,tb9,tba,tbb,fro,fru,frv,fre,xlx,yly,xmu,xba,eps,xkt)

call boundary(rho,rou,rov,roe,nx,ny,uu0,dlt,dx,xlx,yly,xmu,xba,gma)

call adams(rho,rou,rov,roe,fro,gro,fru,gru,frv,grv,&
     fre,gre,nx,ny,dlt)

call state(uuu,vvv,rho,pre,tmp,rou,rov,roe,nx,ny,gma)
```

The result is shown in fig. 8. As you can see this correctly simulates a classical Von Karman vortex street over a cylinder.

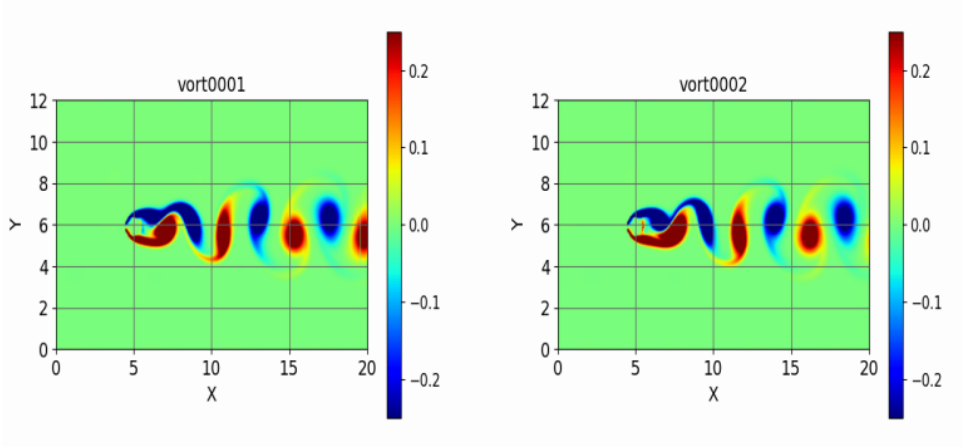

Figure 8: Visualisation of vorticity fields for nt = 9500, 10000, solved using a centred difference scheme in space and a second-order Adams-Bashforth scheme in time with Dirichlet inflow conditions and simple convective equation as outflow condition